Python 28 天 • 分析期

分析期 • 第 10-18 天

清洗、筛选、groupby、merge、透视、时序、可视化

共 9 天：第10天、第11天、第12天、第13天、第14天、第15天、第16天、第17天、第18天

每一份前面的深色页 = 新的一天开始；后面几页就是当天那份知识点 PPT 的原文。

配套：① 开始今天的学习.cmd 每天会自动打开当天那一份；这本合订本是给你周末连着翻、面试前一晚整个过一遍用的。

这本里都有哪几天

第10天 清洗第一课：缺失值、重复行、类型转换、字符串规整
11/11

第11天 筛选与排序：布尔索引、query、lociloc、多列排序
11/12

第12天 groupby 分组聚合：agg、transform、多列分组
11/13

第13天 merge 连接：how 四种取值、连接键对不上怎么办
11/14

第14天 ★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步
11/15

第15天 pivot_table 透视表：indexcolumnsvaluesaggfunc 四件套
11/16

第16天 时间序列：to_datetime、resample、rolling、同比环比
11/17

第17天 applymaplambda + 多级索引：什么时候该用、什么时候别用
11/18

第18天 可视化：matplotlib 中文设置 + 三张有业务含义的图
11/19

分析期 · 第 10 天 · 11/11

清洗第一课：缺失值、重复行、类型转换、字符串规整

把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

第 10 天 • 11/11 • 今日知识点回顾

清洗第一课：缺失值、重复行、类型转换、字符串规整

目标：把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把 1200 行脏订单洗成「类型全对、无重复、缺失有交代」的干净表，并写一份清洗账（每步丢了多少行、为什么）。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 47 分钟（边看边在本机敲一遍）

P32	Pandas2数据清洗与准备	约 15 分钟
P25	Pandas2数据类型转换操作	约 8 分钟
P27	Pandas2数据添加修改删除操作	约 24 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch7.rst`

第七章 缺失数据：isna / dropna / fillna / 插值

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • isna 找缺失，dropna 删，fillna 填 —— 先想清楚删还是填

- 一句话：能算出来的缺失就填（金额=单价×数量），算不出来的才删；每一步都要记账。
- 代码：

```
import pandas as pd, numpy as np, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'orders_dirty.csv', encoding='utf-8-sig')
print(df.isna().sum().to_dict())
print('删掉任一缺失的行：', len(df.dropna()))
print('只删 amount 缺的行：', len(df.dropna(subset=['amount'])))
print('city 填空：', len(df['city'].fillna('（未知）')))
```

知识点 1/7 • isna 找缺失，dropna 删，fillna 填 —— 先想清楚删还是填（续）

★ dropna() 不给参数是「任一行有任一缺失就删」—— 本机这 1200 行里会被删掉 90 行（35 金额 + 56 城市，可能有重叠）。先 dropna(subset=[...])

- 指定列，别一上来就 dropna()。

这一页只讲「isna 找缺失，dropna 删，fillna 填 —— 先想清楚删还是填」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 类型转换：to_numeric(errors='coerce') 是主力

- 一句话：errors='coerce' 把转不了的值变成 nan（而不是抛异常），配合先洗字符串。
- 代码：

```
s = df['amount'].astype(str).str.replace('¥', '', regex=False).str.replace(',', '', regex=False)
df['amount'] = pd.to_numeric(s, errors='coerce')
print(df['amount'].dtype, df['amount'].isna().sum())    # float64 35
# 对照：不加 coerce 会直接抛错
# pd.to_numeric(df['amount'])  →  ValueError: Unable to parse string "¥1,899.00"
```

★ 报错原文：ValueError: Unable to parse string "¥1,899.00" at position 3。

- 「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。

这一页只讲「类型转换：to_numeric(errors='coerce') 是主力」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • 字符串规整三件套：strip / lower / replace

- 一句话：前后空格、大小写、内部空格 —— 清洗状态列时必须三样都做。
- 代码：

```
print(df['status'].unique())                    # 6 种写法
df['status'] = df['status'].astype(str).str.strip().str.lower()
print(df['status'].value_counts().to_dict())
# {'paid': 417, 'pending': 406, 'refunded': 377}
```

★ 'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。

知识点 4/7 • drop_duplicates：先想清楚「重复」的定义

- 一句话：drop_duplicates(subset=['order_id'], keep='first') 按业务主键去重。
- 代码：

```
print('重复单号行数：', df['order_id'].duplicated().sum())      # 37
before = len(df)
df = df.drop_duplicates(subset=['order_id'], keep='first').reset_index(drop=True)
print('去重：', before, '→', len(df))                            # 1200 → 1163
```

★ 不给 subset 是「整行完全相同才删」—— 本机这 1200 行的整行重复是 0 行，所以 df.drop_duplicates() 一行都不会删。业务上说的「重复」几乎都是主键重复，一定要给 subset。

知识点 5/7 • 日期解析：本机这份数据有 145 行中文日期，format='mixed' 救不了

- 一句话：中文日期（2026年8月1日）必须单独用 format='%Y年%m月%d日' 再解析。
- 代码：

```
ct = df['create_time'].astype(str)
print(ct.str.contains('年').sum())                  # 145
a = pd.to_datetime(ct, format='mixed', errors='coerce')
print('mixed 解析失败：', a.isna().sum())            # 145
cn = pd.to_datetime(ct, format='%Y年%m月%d日 %H:%M:%S', errors='coerce')
print('中文格式单独解析成功：', cn.notna().sum())
```

知识点 5/7 • 日期解析：本机这份数据有 145 行中文日期，format='mixed' 救不了...

★ 本机实测：pandas 3.0.6 的 format='mixed' 也不认「2026年8月1日」，

- 145 行全部变 NaT。必须先用 str.replace('年','-').replace('月','-').replace('日','')
- 或单独指定 format。这是「看起来能跑、结果悄悄少了 145 行」的典型。

知识点 6/7 • 日期统一成标准格式：一次 replace 解决

- 一句话：把「年月日」换成「-」，再统一 parse，比写三个分支简单。
- 代码：

```
norm = (ct.str.replace('年', '-', regex=False)
.str.replace('月', '-', regex=False)
.str.replace('日', '', regex=False)
.str.replace('/', '-', regex=False))
df['create_time'] = pd.to_datetime(norm, format='mixed', errors='coerce')
print(df['create_time'].isna().sum())          # 0
print(df['create_time'].min(), df['create_time'].max())
```

- （本机实测：清洗后 1163 行日期全部解析成功，范围 2026-08-01 到 2026-10-29。
- 注意「只有日期没有时间」的 42 行会被补成 00:00:00 —— 这是对的，但如果按小时做分析，要单独标出来。）

知识点 7/7 • 清洗账：每步都记一笔，最后一起打印

- 一句话：用 dict 记 {步骤: 影响行数}，跑完打印 —— 这是能拿去面试讲的「清洗口径」。
- 代码：

```
log = {}
log['原始行数'] = len(df)
```

知识点 7/7 • 清洗账：每步都记一笔，最后一起打印（续）

- log['去重删除'] = 37log['金额无法解析'] = int(df['amount'].isna().sum())
- log['日期无法解析'] = int(df['create_time'].isna().sum())
- log['城市缺失已填'] = int((df['city'] == '（未知）').sum())

```
for k, v in log.items():
print(f'{k:12s}{v:>6d}')
```

★ 面试问「你怎么处理脏数据」时，答「我有 6 条规则，每条影响多少行」比答「我用了 dropna 和 fillna」高一个量级 —— 因为前者证明你量过**。**

代码速查：今天这几段，要在本机跑通（1/8）

写 clean_orders_v2.py，产出 clean_v2.csv + clean_log.json：

```
import json, pathlib
```

```
import pandas as pd
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def clean(path):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/8）

```
    df = pd.read_csv(path, encoding='utf-8-sig')
```

```
log = {'raw_rows': len(df)}

# 1) 金额
```

```
    s = (df['amount'].astype(str).str.replace('¥', '', regex=False)
```

```
           .str.replace(',', '', regex=False))
```

```
    df['amount'] = pd.to_numeric(s, errors='coerce')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/8）

```
    log['amount_na'] = int(df['amount'].isna().sum())
```

```
    # 2) 状态规整
```

```
    df['status'] = df['status'].astype(str).str.strip().str.lower()
```

```
    # 3) 城市填空
```

```
    df['city'] = df['city'].fillna(' (未知) ').str.strip().replace('', ' (未知) ')
```

```
    log['city_filled'] = int((df['city'] == ' (未知) ').sum())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/8）

```
# 4) 日期统一
ct = df['create_time'].astype(str)
norm = (ct.str.replace('年', '-', regex=False)
          .str.replace('月', '-', regex=False)
          .str.replace('日', '', regex=False)
          .str.replace('/', '-', regex=False))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/8）

```
    df['create_time'] = pd.to_datetime(norm, format='mixed', errors='coerce')
```

```
    log['time_na'] = int(df['create_time'].isna().sum())
```

5) 去重

before = len(df)

```
    df = df.drop_duplicates(subset=['order_id'], keep='first').reset_index(drop=True)
```

```
    log['dup_dropped'] = before - len(df)
```

代码速查：今天这几段，要在本机跑通（6/8）

```
    # 6) 数量：负数当录入错误，置空
    neg = int((df['quantity'] < 0).sum())
```

```
    df.loc[df['quantity'] < 0, 'quantity'] = pd.NA
```

```
    log['qty_negative'] = neg
    log['clean_rows'] = len(df)
    return df, log

if __name__ == '__main__':
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/8）

```
df, log = clean(DATA / 'orders_dirty.csv')
```

```
df.to_csv(DATA / 'clean_v2.csv', index=False, encoding='utf-8-sig')
```

```
(DATA / 'clean_log.json').write_text(
```

```
    json.dumps(log, ensure_ascii=False, indent=2), encoding='utf-8')
```

```
print(json.dumps(log, ensure_ascii=False, indent=2))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（8/8）

```
print(df.dtypes)
```

验收（做到才算过关）：

① 六个数字对得上：raw 1200 / amount_na 35 / city_filled 56 / time_na 0 / dup 37 / clean 1163；

② df.dtypes 里 amount 是 float64、create_time 是 datetime64[ns]；

③ 能解释「为什么去重放在日期清洗之后、数量置空之前」——顺序影响 clean_rows。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方（1/2）

指定列，别一上来就 dropna()。

★ 报错原文：ValueError: Unable to parse string "¥1,899.00" at position 3。

「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。

★ 'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

最容易错的地方（2/2）

> 注意「只有日期没有时间」的 42 行会被补成 00:00:00 —— 这是对的，但如果按小时做分析，要单独标出来。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：'paid ' 和 'paid' 在 groupby 里是两组 —— 这一条错，第 12 天的品类汇总会多出一行、金额少一截，而且不报错。
- [ ] 我能不看资料说出：「at position 3」告诉你第 3 行有问题，但真实数据里往往是几千行都有问题 ——所以一律先 coerce 再统计，别指望它一行行报。
- [ ] 我能不看资料说出：一句话：前后空格、大小写、内部空格 —— 清洗状态列时必须三样都做。
- [ ] 我能不看资料说出：一句话：中文日期（2026年8月1日）必须单独用 format='%Y年%m月%d日' 再解析。
- [ ] 我能不看资料说出：145 行全部变 NaT。必须先用 str.replace('年','-').replace('月','-').replace('日','')
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 11 天 • 筛选与排序：布尔索引、query、loc/iloc、多列排序

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-11-第10天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 11 天 · 11/12

筛选与排序：布尔索引、query、lociloc、多列排序

用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

第 11 天 • 11/12 • 今日知识点回顾

筛选与排序：布尔索引、query、loc/iloc、多列排序

目标：用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 37 分钟（边看边在本机敲一遍）

P23	Pandas2数据选择操作	约 18 分钟
P24	Pandas2复杂数据查询操作	约 15 分钟
P28	Pandas2高级查询过滤	约 4 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：loc / iloc / 布尔索引 / 多层索引

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 布尔索引：一行写完所有条件

- 一句话：df[df['列'] > 值] 是核心写法；多个条件用 & | ~，每个条件都要加括号。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
print(len(df[df['amount'] > 2000]))
print(len(df[(df['status'] == 'paid') & (df['city'] == '北京')]))
print(len(df[(df['status'] == 'paid') | (df['amount'] > 5000)]))
print(len(df[~(df['status'] == 'refunded')]))
```

知识点 1/7 • 布尔索引：一行写完所有条件（续）

★ 不加括号会报错：

• ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),a.item(), a.any() or a.all(). 原因是 & 的优先级比 > 高，Python 先算 'paid' & df['city']，整串就乱了。

知识点 2/7 • query：条件写成字符串，适合「条件很多、还要给业务看」

- 一句话：df.query("amount > 2000 and city == '北京'")，用 and/or 不用 &/|。
- 代码：

```
print(len(df.query("amount > 2000")))
print(len(df.query("status == 'paid' and city == '北京'")))
city = '上海'
print(len(df.query("city == @city")))      # 用 @ 引用外部变量
```

★ query 里列名有空格或中文要反引号：df.query("`销售员` == '孙悦'")。

- 本机 clean_v2.csv 的列名是英文，暂时用不上，但读 Excel 时天天遇到。

这一页只讲「query：条件写成字符串，适合「条件很多、还要给业务看」」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • loc 与 iloc：一个按名字、一个按位置

- 一句话：loc[行标签, 列名]；iloc[行号, 列号] —— 行号从 0 开始，不含右端。
- 代码：

```
d2 = df.set_index('order_id')
print(d2.loc[100001, ['city', 'amount']].to_dict())
print(d2.iloc[0:2, 0:3].to_string())
print(df.loc[0:2, 'city'].tolist())        # loc 的切片**含右端**
```

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

- 记法：loc 是「标签切片」，像字典；iloc 是「位置切片」，像 list。

知识点 4/7 • 排序：sort_values 多列 + 稳定排序

- 一句话：df.sort_values(['列1', '列2'], ascending=[True, False])。
- 代码：

```
top = (df.sort_values(['amount', 'order_id'], ascending=[False, True])
.head(5)[['order_id', 'city', 'amount']])
print(top.to_string(index=False))
print(df.nlargest(3, 'amount')[['order_id', 'amount']].to_string(index=False))
```

★ nlargest/nsmallest 比 sort_values().head() 快（内部用堆，不全排），只要前 N 名时优先用它。sort_values 默认 ascending=True，写反了会得到最小的一批。

知识点 5/7 • isin / between / str.contains：三个高频条件写法

- 一句话：isin 判「属于集合」，between 判区间（含两端），str.contains 做模糊匹配。
- 代码：

```
print(len(df[df['city'].isin(['北京', '上海', '广州'])]))
print(len(df[df['amount'].between(1000, 2000)]))
print(len(df[df['product_name'].str.contains('小米', na=False)]))
print(len(df[df['product_name'].str.startswith('罗技', na=False)]))
```

★ str.contains 默认 na=nan 会返回 nan，在布尔索引里被当成 False —— 看着没事，但 df[mask] 的行数会比你算的少。一律显式写 na=False。

知识点 6/7 • 真数据：几个能直接拿去讲的筛选结果

- 一句话：每个筛选都要能说出「筛出来多少行、为什么是这个数」。
- 代码：

```
paid = df[df['status'] == 'paid']
print('paid 行数', len(paid), '合计', round(paid['amount'].sum(), 2))
print('北京 paid', len(paid[paid['city'] == '北京']))
print('金额 1000-2000', len(df[df['amount'].between(1000, 2000)]))
print('品类是数码', len(df[df['category'] == '数码']))
```

- （本机实测 clean_v2.csv：paid 417 行，合计 422,436.00；
- 北京 paid 若干行；金额 1000-2000 的行数可以直接数。
- 注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

这一页只讲「真数据：几个能直接拿去讲的筛选结果」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • 筛选完记得 copy，别改到原表

- 一句话：df[mask] 拿到的是视图或副本不确定，要改就显式 .copy()。
- 代码：

```
sub = df[df['status'] == 'paid'].copy()
```

知识点 7/7 • 筛选完记得 copy，别改到原表（续）

- sub['amount_disc'] = sub['amount'] * 0.9　　# 安全

```
# 不 copy 直接赋值会报 SettingWithCopyWarning
```

★ pandas 3.0 强制开启 Copy-on-Write，SettingWithCopyWarning 基本消失，但「改了子集、原表没变」这个语义更容易让人困惑 —— 显式 .copy() 最稳。

代码速查：今天这几段，要在本机跑通（1/5）

写 filter_practice.py，每题打印行数 + 一行解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) 三种写法写同一个条件：status=paid 且 city=北京

```
a = len(df[(df['status'] == 'paid') & (df['city'] == '北京')])
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
b = len(df.query("status == 'paid' and city == '北京'"))
```

```
c = len(df.loc[(df['status'] == 'paid') & (df['city'] == '北京')])
```

```
print('三种写法：', a, b, c)          # 三个数必须相同
```

2) 金额 Top 10，用 nlargest

```
print(df.nlargest(10, 'amount')[['order_id', 'city', 'category', 'amount']].to_string(index=False))
```

3) 区间 + 集合 + 模糊

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
print('1000-2000: ', len(df[df['amount'].between(1000, 2000)]))
```

```
print('北上广: ', len(df[df['city'].isin(['北京', '上海', '广州'])]))
```

```
print('商品名含 小米: ', len(df[df['product_name'].str.contains('小米', na=False)]))
```

4) 排除退款

```
print('非退款行数: ', len(df[df['status'] != 'refunded']))
```

5) 按两列排序取前 5

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
print(df.sort_values(['city', 'amount'], ascending=[True, False])
```

```
        .groupby('city').head(1)[['city', 'order_id', 'amount']]
```

.to_string(index=False))

验收（做到才算过关）：

① 第 1 题三个数完全一致；

```
② 能说清为什么 (df['a'] > 1) & (df['b'] < 2) 不加括号会报 ValueError；
```

代码速查：今天这几段，要在本机跑通（5/5）

```
③ 第 5 题能解释 groupby('city').head(1) 和直接 head(1) 的差别。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料说出：注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 12 天 • groupby 分组聚合：agg、transform、多列分组

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-12-第11天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 12 天 · 11/13

groupby 分组聚合：agg、transform、多列分组

给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

第 12 天 • 11/13 • 今日知识点回顾

groupby 分组聚合：agg、transform、多列分组

目标：给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 给出「每个品类每个城市的实付合计与订单数」，并说清 agg 和 transform 的结果形状差别。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 37 分钟（边看边在本机敲一遍）

P34	Pandas2数据分组与聚合	约 6 分钟
P33	【数据的合并和分组聚合】03数据分组聚合	约 15 分钟
P34	【数据的合并和分组聚合】04数据分组聚合02	约 16 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch4.rst`

第四章 分组：groupby 的三大操作（聚合/变换/过滤）

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • groupby 三步：拆、算、合

- 一句话：df.groupby('列')['要算的列'].agg(函数) —— 拆开、每块算、拼回来。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
paid = df[df['status'] == 'paid']
g = paid.groupby('category')['amount'].agg(['count', 'sum', 'mean']).round(2)
print(g.sort_values('sum', ascending=False).to_string())
```

这一页只讲「groupby 三步：拆、算、合」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • groupby 三步：拆、算、合（续）

★ 结果（本机实测）：数码 132 单 / 192,231.00 / 均价 1,456.30；

- 家居 113 单 / 147,895.00 / 1,308.81；美妆 74 单 / 59,704.00 / 806.81；
- 食品 85 单 / 22,606.00 / 265.95。四个数加起来 = 422,436.00 = 昨天的 paid 合计。

知识点 2/7 • as_index=False 让分组键留在列里

- 一句话：groupby(..., as_index=False) 等价于 .reset_index()，少写一步。
- 代码：

```
t = paid.groupby('city', as_index=False)['amount'].sum().sort_values('amount', ascending=False)
print(t.head(3).to_string(index=False))
print(t.columns.tolist())      # ['city', 'amount']
```

★ 不加 as_index=False 时，city 变成索引，后面 to_csv 会丢一列 ——这是「导出的文件少一列」最常见的原因。

知识点 3/7 • 命名聚合：一个列一次算多个指标，输出列名自己定

- 一句话：agg(总单数=('order_id', 'count'), 实付=('amount', 'sum'))。
- 代码：

```
r = paid.groupby('city').agg(
总单数=('order_id', 'count'),
实付合计=('amount', 'sum'),
客单价=('amount', 'mean'),
最大单=('amount', 'max'),
).round(2).sort_values('实付合计', ascending=False)
print(r.head(5).to_string())
```

★ 命名聚合要求 pandas ≥ 0.25，本机 3.0.6 完全支持。老写法agg({'amount': ['sum', 'mean']}) 会得到多级列名，导出 CSV 后表头是两行 ——给业务看时一律用命名聚合。

知识点 4/7 • transform：算完把结果「铺回原表的每一行」

- 一句话：agg 每个组返回一行；transform 每个组返回和原组一样多的行。
- 代码：

```
df['city_total'] = df.groupby('city')['amount'].transform('sum')
df['city_share'] = (df['amount'] / df['city_total'] * 100).round(2)
print(df[['order_id', 'city', 'amount', 'city_total', 'city_share']].head(5).to_string(index=False))
```

★ 用 agg 做这件事要先 merge 回去，transform 一步到位 —— 算「占比」时必用。

- 注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。

知识点 5/7 • 多列分组 + 交叉表

- 一句话：groupby(['城市', '品类']) 得到多层索引，unstack() 摊成二维表。
- 代码：

```
pv = (paid.groupby(['city', 'category'])['amount'].sum()
.unstack(fill_value=0).round(0))
print(pv.shape)                                    # (9, 4)
print(pv.sum(axis=1).sort_values(ascending=False).head(3).to_string())
```

★ 本机实测：9 个城市 × 4 个品类，实付合计最高的城市是南京（60,532.00）。

- 这个结果第 15 天用 pivot_table 会再算一次，两个数必须一致。

知识点 6/7 • size 与 count 的差别（老坑）

- 一句话：size 数「行数（含 nan）」，count 数「非 nan 的个数」。
- 代码：

```
print(paid.groupby('city').size().head(3).to_string())
print(paid.groupby('city')['amount'].count().head(3).to_string())
```

★ 只要有一个金额是 nan，这两个数就不一样。回答「一共多少单」时，永远用 size() 或 count 在非空列上 —— 否则会少算。

知识点 7/7 · groupby 的两个性能与正确性开关

- 一句话：observed=True（分类列只保留出现过的组合）、sort=False（不排序更快）。
- 代码：

```
df['city'] = df['city'].astype('category')
print(len(df.groupby(['city', 'category'], observed=True)))
print(len(df.groupby(['city', 'category'], observed=False)))   # 笛卡尔积，可能更大
```

这一页只讲「groupby 的两个性能与正确性开关」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 · groupby 的两个性能与正确性开关（续）

- （pandas 3.0 的 groupby 默认 observed=True 了，但显式写出来更保险；

```
observed=False 时分类列会补齐所有组合，行数暴涨且大量为 0 —
```

- 这是「分组结果莫名其妙多了几百行」的元凶。）

代码速查：今天这几段，要在本机跑通（1/5）

写 groupby_practice.py，产出五个能直接贴进报告的表：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

```
# 1）品类汇总（对照本机数字：数码 192231 / 家居 147895 / 美妆 59704 / 食品 22606）
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print(paid.groupby('category').agg(
```

```
    单数=('order_id', 'count'), 实付=('amount', 'sum'), 客单价=('amount', 'mean')
```

```
).round(2).sort_values('实付', ascending=False).to_string())
```

2) 城市汇总，as_index=False

```
print(paid.groupby('city', as_index=False)['amount'].sum()
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
          .sort_values('amount', ascending=False).head(5).to_string(index=False))
```

3) 城市 × 品类 交叉（unstack）

```
print(paid.groupby(['city', 'category'])['amount'].sum()
```

```
          .unstack(fill_value=0).round(0).to_string())
```

4) 每个订单占所在城市实付的百分比（transform）

```
df['city_total'] = df.groupby('city')['amount'].transform('sum')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
df['share'] = (df['amount'] / df['city_total'] * 100).round(2)
```

```
print(df.nlargest(5, 'share')[['order_id', 'city', 'amount', 'city_total', 'share']]
```

.to_string(index=False))

```
# 5) 每个城市的 Top1 订单（groupby + nlargest 的组合写法）
```

```
print(paid.sort_values('amount', ascending=False)
```

代码速查：今天这几段，要在本机跑通（5/5）

```
        .groupby('city').head(1)[['city', 'order_id', 'amount']]
```

```
        .sort_values('amount', ascending=False).to_string(index=False))
```

验收（做到才算过关）：

① 第 1 题四个品类的实付合计相加 = 422,436.00；

② 第 3 题交叉表是 9 行 × 4 列，合计最高的城市是南京；

```
③ 能说清 transform 和 agg 的结果形状差别，并各举今天代码里的一处用法。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

> 注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：不加 as_index=False 时，city 变成索引，后面 to_csv 会丢一列 ——这是「导出的文件少一列」最常见的原因。
- [ ] 我能不看资料写出：用 agg 做这件事要先 merge 回去，transform 一步到位 —— 算「占比」时必用。
- [ ] 我能不看资料写出：只要有一个金额是 nan，这两个数就不一样。回答「一共多少单」时，永远用 size() 或 count 在非空列上 —— 否则会少算。
- [ ] 我能不看资料说出：注意分母含 nan 时结果是 nan，要先用 fillna(0) 或只对 paid 算。
- [ ] 我能不看资料说出：这个结果第 15 天用 pivot_table 会再算一次，两个数必须一致。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 13 天 • merge 连接：how 四种取值、连接键对不上怎么办

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-13-第12天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 13 天 · 11/14

merge 连接：how 四种取值、连接键对不上怎么办

能解释「为什么 left join 之后行数变多了」。

第 13 天 • 11/14 • 今日知识点回顾

merge 连接：how 四种取值、连接键对不上怎么办

目标：说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 说清 left / inner / outer / right 四种连接的差别，并能解释「为什么 left join 之后行数变多了」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 52 分钟（边看边在本机敲一遍）

P31	Pandas2数据合并与对比	约 21 分钟
P32	【数据的合并和分组聚合】02数据合并	约 19 分钟
P35	【数据的合并和分组聚合】05数据的索引学习	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch6.rst`

第六章 连接：merge / join / concat 的区别与 on/left_on/right_on

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • merge 的四个 how，差别只在「留谁的行」

- 一句话：inner 交集、left 留左表全部、right 留右表全部、outer 并集。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
for how in ['inner', 'left', 'right', 'outer']:
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how=how)
print(how, len(m), m['gender'].isna().sum())
```

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

这一页只讲「merge 的四个 how，差别只在「留谁的行」」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • left join 行数变多 = 右表在连接键上不唯一

- 一句话：右表一个键对应多行，左表一行就会被复制成多行 —— 这是最常见的「数据翻倍」事故。
- 代码：

```
print('右表 user_id 唯一? ', u['user_id'].is_unique)      # True
dup = pd.DataFrame({'user_id': [1001, 1001], 'tag': ['新客', '老客']})
m = df.head(3).merge(dup, on='user_id', how='left')
print(len(m))                                             # 4 — 1001 那一行变两行
```

★ 连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。

- 多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

这一页只讲「left join 行数变多 = 右表在连接键上不唯一」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • 连接键名字不同：left_on / right_on

- 一句话：左表叫 user_id、右表叫 uid 时，用 left_on='user_id', right_on='uid'。
- 代码：

```
u2 = u.rename(columns={'user_id': 'uid'})
m = df.merge(u2[['uid', 'gender']], left_on='user_id', right_on='uid', how='left')
print(m.columns.tolist()[:6])
print(len(m))
```

★ 这样连完 user_id 和 uid 两列都在，值一样 —— 记得 drop 掉一列，否则后面groupby 会出现两列重复的键，导出 CSV 时业务方会问「怎么有两个 ID」。

知识点 4/7 • indicator 参数：一眼看出哪行没连上

- 一句话：merge(..., indicator=True) 会多一列 _merge，值是 both / left_only / right_only。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
print(m['_merge'].value_counts().to_dict())
miss = m[m['_merge'] == 'left_only']
print('连不上的订单数：', len(miss), '涉及金额：', round(miss['amount'].sum(), 2))
```

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

知识点 5/7 • validate：让 merge 主动检查「一对几」

- 一句话：validate='one_to_one' / 'one_to_many' / 'many_to_one'，不符合直接报错。
- 代码：

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left',
validate='many_to_one')       # 左多右一，符合预期
print(len(m))
# validate='one_to_one' 会报
# MergeError: Merge keys are not unique in either left or right dataset
```

★ 加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。

这一页只讲「validate：让 merge 主动检查「一对几」」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • concat：纵向摞，不是横向连

- 一句话：pd.concat([df1, df2], ignore_index=True) 把两个同结构的表摞起来。
- 代码：

```
a = df.head(3)
b = df.tail(3)
print(len(pd.concat([a, b], ignore_index=True)))      # 6
print(pd.concat([a, b]).index.tolist())                # 0,1,2,1160,1161,1162 重复
```

★ 不加 ignore_index=True，索引会重复（0,1,2,1160,1161,1162），后面df.loc[0] 会返回两行 —— 「莫名其妙多了几行」的另一个来源。

知识点 7/7 • 真数据：把用户信息接到订单上，做分性别汇总

- 一句话：merge 之后 groupby 才有意义 —— 单表只能按已有列分组。
- 代码：

```
m = df.merge(u[['user_id', 'gender', 'age']], on='user_id', how='left')
paid = m[m['status'] == 'paid']
g = (paid.groupby('gender', dropna=False)
.agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
.round(2))
print(g.to_string())
```

- （users 表的 gender 有六种写法：男 / 女 / M / F / 未知 / 空。
- 直接 groupby 会得到六行 —— 第 16 天会专门讲怎么归并成两行 + 一行未知。）

这一页只讲「真数据：把用户信息接到订单上，做分性别汇总」。讲完就翻页，下一页是另一个知识点。

代码速查：今天这几段，要在本机跑通（1/5）

写 merge_practice.py，每题打印行数并解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

1) 四种 how 的行数与缺失数

代码速查：今天这几段，要在本机跑通（2/5）

```
for how in ['inner', 'left', 'right', 'outer']:
```

```
    m = df.merge(u[['user_id', 'gender']], on='user_id', how=how)
```

```
    print(f'{how:6s} rows={len(m):5d} gender_na={m["gender"].isna().sum():4d}')
```

2) indicator 找出连不上的订单

```
m = df.merge(u[['user_id', 'gender']], on='user_id', how='left', indicator=True)
```

代码速查：今天这几段，要在本机跑通（3/5）

```
print(m['_merge'].value_counts().to_dict())
```

miss = m[m['_merge'] == 'left_only']

```
print('连不上：', len(miss), '金额合计', round(miss['amount'].sum(), 2))
```

3) validate 检查一对多

```
ok = df.merge(u[['user_id', 'gender']], on='user_id', how='left', validate='many_to_one')
```

```
print('many_to_one 通过，行数', len(ok))
```

代码速查：今天这几段，要在本机跑通（4/5）

```
# 4) 接上用户表后按性别汇总
paid = ok[ok['status'] == 'paid']
print(paid.groupby('gender', dropna=False)
          .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
          .round(2).to_string())
```

验收（做到才算过关）：

```
① 第 1 题四个 how 的行数都能解释（inner < left ≤ outer）；
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② 连不上的订单数是 39，金额合计能报出来；

③ 能说清「为什么 left join 可能比左表行数多」，并举一个具体例子。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 本机实测：left 连接后 1163 行，其中 39 行的 gender 是 nan ——说明订单里有 39 个 user_id 在 users 表里查不到。这 39 行就是「脏数据」，清洗时要单独交代（是用户被删了，还是订单表的 user_id 写错了）。

多对多连接会让汇总金额翻倍，而且不报错 —— 这是最贵的一类 bug。

★ 连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：连接前先跑 df['键'].is_unique 和 df['键'].duplicated().sum()。
- [ ] 我能不看资料写出：连不上的行不是「小事」—— 如果它们占了 5% 的金额，任何按用户维度的分析都会系统性偏低。报数字，别只说「有少量缺失」。
- [ ] 我能不看资料写出：加上 validate 之后，「不小心连成多对多」会在当场炸掉，而不是三天后发现报表数字翻倍。生产脚本里建议每个 merge 都带。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 14 天 • ★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-14-第13天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 • 第 14 天 • 11/15

★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

第 14 天 • 11/15 • 今日知识点回顾

★W5 总验收：给一份脏 CSV，独立清洗成可分析表并说清每一步

目标：90 分钟内从 orders_dirty.csv 走到一份能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 90 分钟内从 orders_dirty.csv 走到一份能交给别人的汇总表 + 一段 200 字的清洗说明，中途不查教程。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 20 分钟（边看边在本机敲一遍）

P28	Pandas2高级查询过滤	约 4 分钟
P29	Pandas2数据迭代操作	约 6 分钟
P30	Pandas2高级函数应用	约 10 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch1.rst`

回看第一章「列表推导式」那一节，检查自己是不是还在写 for + append

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/5 • 验收的判据是「能复现」，不是「跑通一次」

- 一句话：同一份脚本换个数据目录还能跑出结果，才算真会；只在自己电脑上跑通一次不算。
- 代码：

```
import pathlib, sys
DATA = pathlib.Path(sys.argv[1]) if len(sys.argv) > 1 else pathlib.Path(
r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
print('数据目录：', DATA, DATA.exists())
```

★ 把路径写死在代码里，是「换个环境就跑不了」的第一大原因。

- 今天所有脚本都改成命令行参数或环境变量。

知识点 2/5 • 清洗口径要写成「可复查的数字」，不是形容词

- 一句话：不说「有很多重复」，说「37 个重复单号，按 first 保留」。
- 代码：

```
LOG = {
'raw_rows': 1200,
'dup_order_id': 37,
'amount_unparsable': 35,
'city_missing': 56,
'time_cn_format': 145,
'qty_negative': 25,
'clean_rows': 1163,
```

这一页只讲「清洗口径要写成「可复查的数字」，不是形容词」。讲完就翻页，下一页是另一个知识点。

知识点 2/5 · 清洗口径要写成「可复查的数字」，不是形容词（续）

- }

```
for k, v in LOG.items():
print(f'{k:20s}{v:>6d}')
```

★ 这七个数字就是今天要背下来的「清洗口径」。面试被问「你洗过什么脏数据」时，报数字比报函数名有说服力。

知识点 3/5 • 汇总表要能回答一个业务问题

- 一句话：每个输出表都要对应一句「谁看了会做什么决定」。
- 代码：

```
paid = df[df['status'] == 'paid']
out = (paid.groupby(['city', 'category'])
.agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
.round(2).reset_index())
print(out.sort_values('实付', ascending=False).head(8).to_string(index=False))
```

★ 「城市 × 品类」这张表的业务问题：哪些城市在哪个品类上还有空间?

- 南京 60,532 排第一，但如果它只在数码上强、家居弱，那就是铺货问题。

知识点 4/5 • 验收脚本的结构：一个文件、四段、能重跑

- 一句话：load / clean / analyze / report 四段，main 里只做编排，每段可单独测。
- 代码：

```
def main(argv):
data_dir = pathlib.Path(argv[1]) if len(argv) > 1 else DEFAULT
raw = load(data_dir / 'orders_dirty.csv')
clean, log = clean(raw)
tables = analyze(clean)
report(clean, tables, log, data_dir / 'w5_report')
print('完成：', log)
```

- （今天产出的 w5_report/ 目录里应该有：clean.csv、summary_city_category.csv、summary_category.csv、clean_log.json —— 四个文件，缺一个都算没做完。）

知识点 5/5 • 自查清单：这 8 条全过才算 W5 过关

- 一句话：逐条打勾，别靠「感觉学会了」。
- 代码：

```
CHECK = [
'能不看资料写出 groupby + agg + 命名聚合',
'能说清 agg 与 transform 的结果形状差别',
'能说清 merge 的 how 四种取值的差别',
'能解释为什么 loc[0:2] 是 3 行、iloc[0:2] 是 2 行',
'能把三种日期格式统一成 datetime 且不丢行',
'能写出 6 条清洗规则并报出每一条影响多少行',
'能用 pivot_table 做一张城市 × 品类的交叉表',
'能给出一份 200 字的清洗说明，不含形容词只有数字',
```

知识点 5/5 • 自查清单：这 8 条全过才算 W5 过关（续）

-]

★ 8 条里有任何一条说不清，明天补 —— W6 的 pivot_table 和时间序列都建立在这上面。

这一页只讲「自查清单：这 8 条全过才算 W5 过关」。讲完就翻页，下一页是另一个知识点。

代码速查：今天这几段，要在本机跑通（1/7）

独立完成 w5_report.py（不许抄前面的代码），跑出四个文件：

结构示意，具体实现自己写

```
import json, pathlib, sys
```

```
import pandas as pd
```

```
DEFAULT = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def load(path):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/7）

```
    return pd.read_csv(path, encoding='utf-8-sig')
```

```
def clean(df):
```

log = {}

```
    # 金额 / 状态 / 城市 / 日期 / 去重 / 数量 —— 六条规则，每条都记数
```

...

return df, log

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/7）

```
def analyze(df):
```

```
    paid = df[df['status'] == 'paid']
    return {
        'city_category': paid.groupby(['city', 'category'])
                            .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'))
                            .round(2).reset_index(),
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/7）

```
        'category': paid.groupby('category')
```

```
                    .agg(单数=('order_id', 'count'), 实付=('amount', 'sum'),
```

```
                         客单价=('amount', 'mean'))
```

```
                    .round(2).reset_index(),
```

```
    }
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/7）

```
def report(df, tables, log, outdir):
```

```
    outdir.mkdir(parents=True, exist_ok=True)
```

```
    df.to_csv(outdir / 'clean.csv', index=False, encoding='utf-8-sig')
```

for k, t in tables.items():

```
        t.to_csv(outdir / f'summary_{k}.csv', index=False, encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/7）

```
    (outdir / 'clean_log.json').write_text(
```

```
        json.dumps(log, ensure_ascii=False, indent=2), encoding='utf-8')
```

```
if __name__ == '__main__':
```

```
    d = pathlib.Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT
```

```
raw = load(d / 'orders_dirty.csv')

clean_df, log = clean(raw)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/7）

```
    report(clean_df, analyze(clean_df), log, d / 'w5_report')
```

```
print(log)
```

验收（做到才算过关，这是 W5 的总验收）：

① 四个文件都产出，clean.csv 是 1163 行；

② 品类汇总四个数字相加 = 422,436.00；

③ 200 字清洗说明能背下来讲，只含数字不含形容词；

④ 把脚本里的 DEFAULT 改成一个不存在的目录再跑一次，报错信息要能看懂。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：把路径写死在代码里，是「换个环境就跑不了」的第一大原因。
- [ ] 我能不看资料写出：这七个数字就是今天要背下来的「清洗口径」。面试被问「你洗过什么脏数据」时，报数字比报函数名有说服力。
- [ ] 我能不看资料说出：「城市 × 品类」这张表的业务问题：哪些城市在哪个品类上还有空间？
- [ ] 我能不看资料写出：8 条里有任何一条说不清，明天补 —— W6 的 pivot_table 和时间序列都建立在这上面。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 15 天 • pivot_table 透视表：index/columns/values/aggfunc 四件套

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-15-第14天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 15 天 · 11/16

pivot_table 透视表：indexcolumnsvaluesaggfunc 四件套

不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

第 15 天 • 11/16 • 今日知识点回顾

pivot_table 透视表：index/columns/values/aggfunc 四件套

目标：不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 33 分钟（边看边在本机敲一遍）

P33	Pandas2数据重塑透视	约 18 分钟
P33	【数据的合并和分组聚合】03数据分组聚合	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch5.rst`

第五章 变形：pivot / pivot_table / melt / crosstab 全在这一章

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表

- 一句话：index 放行、columns 放列、values 放要算的数、aggfunc 放怎么算。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
paid = df[df['status'] == 'paid']
pv = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc='sum', fill_value=0)
print(pv.shape)          # (9, 4)
print(pv.round(0).to_string())
```

这一页只讲「pivot_table = groupby + unstack，一步出二维表」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表（续）

★ 本机实测：9 个城市 × 4 个品类，行合计最高的是南京 60,532.00。

• 这个数必须和昨天 groupby(['city','category']).sum().unstack() 的结果完全一致 ——不一致说明有一个用了不同的筛选条件。

知识点 2/7 • aggfunc 可以给多个，也可以给 dict 分别指定

- 一句话：aggfunc=['sum','count'] 出多级列；aggfunc={'amount':'sum','order_id':'count'} 按列分。
- 代码：

```
pv2 = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc=['sum', 'count'], fill_value=0)
print(pv2.columns.tolist()[:4])
pv3 = pd.pivot_table(paid, index='city', columns='category',
values=['amount', 'order_id'],
aggfunc={'amount': 'sum', 'order_id': 'count'}, fill_value=0)
print(pv3.shape)
```

★ 多级列名导出 CSV 后表头是两行，Excel 打开看着乱。给业务看时用

- pv.columns = [f'{a}_{b}' for a, b in pv.columns] 拍平，或者只算一个指标。

知识点 3/7 • margins=True 加一行一列合计

- 一句话：margins=True 加「All」行列，margins_name 改名字。
- 代码：

```
pv4 = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
print(pv4.round(0).to_string())
print('总计：', round(pv4.loc['合计', '合计'], 2))      # 422436.00
```

★ 422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。

- 三次都对上，才说明你对「paid」的定义没变。

知识点 4/7 • fill_value=0 与 dropna 的取舍

- 一句话：没成交的组合到底算 0 还是算「没有」——这是业务问题，不是技术问题。
- 代码：

```
a = pd.pivot_table(paid, index='city', columns='category', values='amount', aggfunc='sum')
print('不填: ', a.isna().sum().sum(), '个空')
b = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0)
print('填 0: ', b.isna().sum().sum(), '个空')
```

★ 算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。

知识点 5/7 • crosstab：专做「计数」的透视表，比 pivot_table 短

- 一句话：pd.crosstab(df['city'], df['category']) 直接数每个组合多少单。
- 代码：

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
print(ct.to_string())
ct2 = pd.crosstab(paid['city'], paid['category'], values=paid['amount'],
aggfunc='sum', margins=True, margins_name='合计')
print(ct2.round(0).to_string())
```

★ crosstab 默认 aggfunc='count'（数行数），pivot_table 默认 'mean' ——默认值不同是新手最容易踩的坑：pivot_table 忘了写 aggfunc 会得到平均数。

这一页只讲「crosstab：专做「计数」的透视表，比 pivot_table 短」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • pivot 和 pivot_table 的区别

- 一句话：pivot 只重塑、不聚合（索引必须唯一）； pivot_table 会聚合（索引可重复）。
- 代码：

```
small = paid.head(3)[['city', 'category', 'amount']]
print(small.pivot(index='city', columns='category', values='amount').to_string())
# 有重复组合时 pivot 报：ValueError: Index contains duplicate entries, cannot reshape
```

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

这一页只讲「pivot 和 pivot_table 的区别」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • melt：透视表的逆操作，宽表变长表

- 一句话：pd.melt(pv.reset_index(), id_vars='city') 把列名变成一列「品类」。
- 代码：

```
long = pd.melt(pv.reset_index(), id_vars='city',
var_name='category', value_name='amount')
print(long.shape, long.head(3).to_string(index=False))
```

- （9 城市 × 4 品类 = 36 行长表。做 BI 看板时几乎都要长表，Tableau 尤其吃长表 —— 第 24 天会用到这个转换。）

代码速查：今天这几段，要在本机跑通（1/6）

写 pivot_practice.py，产出四张表并存 CSV：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

```
out = DATA / 'pivot_out'; out.mkdir(exist_ok=True)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

1) 城市 × 品类 实付矩阵（9×4）

```
pv = pd.pivot_table(paid, index='city', columns='category', values='amount',
```

```
                    aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
```

```
pv.round(2).to_csv(out / 'city_category_amount.csv', encoding='utf-8-sig')
```

```
print(pv.round(0).to_string())
```

代码速查：今天这几段，要在本机跑通（3/6）

```
print('总计', round(pv.loc['合计', '合计'], 2))     # 422436.00
```

2) 城市 × 品类 单数矩阵（用 crosstab）

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
```

```
ct.to_csv(out / 'city_category_count.csv', encoding='utf-8-sig')
```

```
print(ct.to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/6）

```
# 3) 城市 × 状态 实付（含 pending/refunded，看退款占比）
```

```
pv3 = pd.pivot_table(df, index='city', columns='status', values='amount',
```

```
                     aggfunc='sum', fill_value=0)
```

```
pv3['退款占比%'] = (pv3['refunded'] / pv3.sum(axis=1) * 100).round(1)
```

```
print(pv3.round(0).to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

4) 宽表转长表

```
long = pd.melt(pv.reset_index().query("city != '合计'"), id_vars='city',
```

```
               var_name='category', value_name='amount')
```

long = long.query("category != '合计'")

```
long.to_csv(out / 'city_category_long.csv', index=False, encoding='utf-8-sig')
```

```
print('长表', long.shape)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

验收（做到才算过关）：

① 第 1 题 9×4 且总计 = 422,436.00；

② 第 3 题能说出哪个城市的退款占比最高、是多少；

③ 能解释 pivot_table 不写 aggfunc 会得到什么（默认 mean），以及为什么危险。

最容易错的地方

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。
- [ ] 我能不看资料写出：算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。
- [ ] 我能不看资料说出：一句话：pivot 只重塑、不聚合（索引必须唯一）；pivot_table 会聚合（索引可重复）。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 16 天 • 时间序列：to_datetime、resample、rolling、同比环比

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-16-第15天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 16 天 · 11/17

时间序列：to_datetime、resample、rolling、同比环比

把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

第 16 天 • 11/17 • 今日知识点回顾

时间序列：to_datetime、resample、rolling、同比环比

目标：把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把订单按周、按月各汇总一次，并算出环比增长率；对 quant_returns 算滚动 20 日波动率。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 48 分钟（边看边在本机敲一遍）

P35	Pandas2时间序列数据	约 16 分钟
P36	Pandas2数据可视化	约 20 分钟
P37	【案例】时间序列	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch8.rst`

第八章 文本数据（开头讲 to_datetime 的部分先看）

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 时间列必须是 datetime64，否则 resample 用不了

- 一句话：先 to_datetime，再 set_index，最后 resample —— 顺序不能乱。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
print(df['create_time'].dtype)            # datetime64[ns]
print(df['create_time'].min(), df['create_time'].max())
```

★ 本机实测范围：2026-08-01 到 2026-10-29，共 90 天。

- 如果 dtype 是 object，resample 会报 TypeError: Only valid with DatetimeIndex,TimedeltaIndex or PeriodIndex —— 看到这个报错就是没转 datetime。

这一页只讲「时间列必须是 datetime64，否则 resample 用不了」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • resample：按周/月重采样，等价于「按时间分组的 groupby」

- 一句话：df.set_index('时间').resample('W')['金额'].sum()。
- 代码：

```
s = df.set_index('create_time')
weekly = s.resample('W')['amount'].sum().round(2)
print(weekly.head().to_string())
monthly = s.resample('ME')['amount'].sum().round(2)      # ME = 月末
print(monthly.to_string())
```

★ pandas 3.0 里 'M' 已弃用，必须写 'ME'（月末）或 'MS'（月初），否则告警

- FutureWarning: 'M' is deprecated, use 'ME'。同理 'Y'→'YE'、'Q'→'QE'。

知识点 3/7 • 环比与同比：pct_change 一行搞定

- 一句话：s.pct_change() 算相邻两期增长率；同比要看 12 期前，用 shift(12)。
- 代码：

```
monthly = s.resample('ME')['amount'].sum()
mom = (monthly.pct_change() * 100).round(2)
print(pd.DataFrame({'实付': monthly.round(0), '环比%': mom}).to_string())
print('首月无环比：', mom.isna().sum())
```

★ 第一期一定是 nan（没有上一期）。填 0 会让「首月增长 0%」这个假结论进报告 —— 保留 nan，或者在报告里写「首月为基期」。

知识点 4/7 • rolling：滑动窗口，算移动平均与滚动波动率

- 一句话：s.rolling(20).std() 是「最近 20 期」的标准差。
- 代码：

```
q = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig', parse_dates=['date'])
q = q.set_index('date')
```

这一页只讲「rolling：滑动窗口，算移动平均与滚动波动率」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • rolling：滑动窗口，算移动平均与滚动波动率（续）

- q['vol20'] = q['strategy_ret'].rolling(20).std() * (252 ** 0.5) * 100

```
print(q[['strategy_ret', 'vol20']].dropna().head().round(4).to_string())
print('最新 20 日年化波动%: ', round(q['vol20'].dropna().iloc[-1], 2))
```

★ 前 19 行一定是 nan（窗口没满）。rolling(20).std() 默认 ddof=1（样本标准差），和 numpy 的 np.std 默认 ddof=0 不一样 —— 两套算出来的数会有细微差别。

知识点 5/7 • quant 数据实测：356 行、总收益 89.53%

- 一句话：本机 quant_returns.csv 是 2025-01 起的日频回测，策略总收益 89.53%，基准 62.37%。
- 代码：

```
q2 = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig', parse_dates=['date'])
print(q2.shape, q2['date'].min(), q2['date'].max())     # (356,)
total = (1 + q2['strategy_ret']).prod() - 1
bench = (1 + q2['bench_ret']).prod() - 1
print('策略', round(total * 100, 2), '%  基准', round(bench * 100, 2), '%')
```

- （本机实测：356 个交易日，策略总收益 89.53%、基准 62.37%；
- 最大回撤 -18.59%、Sharpe 2.73、胜率 57.0%、年化波动 17.15%。
- 注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。）

知识点 6/7 • 日期抽取：dt 访问器

- 一句话：s.dt.year / .month / .dayofweek / .hour 直接从 datetime 列取成分。
- 代码：

```
d = df['create_time']
print(d.dt.year.value_counts().to_dict())
print(d.dt.month.value_counts().sort_index().to_dict())
print(d.dt.dayofweek.value_counts().sort_index().to_dict())   # 0=周一
df['is_weekend'] = d.dt.dayofweek >= 5
```

★ dt.dayofweek 是 0=周一 … 6=周日（不是 1=周一）。dt.day 是「几号」不是「星期几」，这两个搞混会算出一堆没意义的「按星期分布」。

知识点 7/7 • 时间对齐：两个序列要按同一个日历合并

- 一句话：reindex 到统一日期轴，缺的补 nan 或前值填充。
- 代码：

```
a = q['strategy_ret'].resample('ME').sum()
b = q['bench_ret'].resample('ME').sum()
cmp = pd.concat([a.rename('策略'), b.rename('基准')], axis=1)
print(cmp.round(4).to_string())
print('月胜率: ', round((cmp['策略'] > cmp['基准']).mean() * 100, 1), '%')
```

★ 直接 pd.concat 两个不同长度的 Series 会按索引并集对齐，缺的地方是 nan ——不会报错，但会让「月胜率」的分母变大。先 resample 到同一频率再比。

代码速查：今天这几段，要在本机跑通（1/6）

写 timeseries_practice.py，产出四份时间维度的表：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
out = DATA / 'ts_out'; out.mkdir(exist_ok=True)
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

```
paid = df[df['status'] == 'paid'].set_index('create_time')
```

1) 周汇总 + 环比

```
weekly = paid['amount'].resample('W').sum()
```

```
print(pd.DataFrame({'周实付': weekly.round(0), '环比%': (weekly.pct_change() * 100).round(1)}).to_string())
```

2) 月汇总（ME）

```
monthly = paid['amount'].resample('ME').sum()
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/6）

```
monthly.to_csv(out / 'monthly.csv', encoding='utf-8-sig')
```

```
print(monthly.round(2).to_string())
```

3) 按星期几看订单分布

```
print(df['create_time'].dt.dayofweek.value_counts().sort_index().to_string())
```

4) quant：滚动 20 日年化波动 + 月度策略 vs 基准

```
q = pd.read_csv(DATA / 'quant_returns.csv', encoding='utf-8-sig',
```

代码速查：今天这几段，要在本机跑通（4/6）

```
                parse_dates=['date']).set_index('date')
```

```
q['vol20'] = q['strategy_ret'].rolling(20).std() * (252 ** 0.5) * 100
```

cmp = pd.concat([

```
    q['strategy_ret'].resample('ME').sum().rename('策略'),
```

```
    q['bench_ret'].resample('ME').sum().rename('基准'),
```

], axis=1)

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

```
cmp['超额'] = (cmp['策略'] - cmp['基准']).round(4)
```

```
cmp.round(4).to_csv(out / 'monthly_strategy_vs_bench.csv', encoding='utf-8-sig')
```

```
print(cmp.round(4).to_string())
```

```
print('最新年化波动%: ', round(q['vol20'].dropna().iloc[-1], 2))
```

验收（做到才算过关）：

① 周汇总第一行是 nan 环比（不是 0）；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

② 月度表能报出三个月各多少、哪个月最高；

```
③ quant 月度表里「策略」列相加 ≈ 89.53%（累乘口径），能解释为什么月度和 ≠ 总收益。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

如果 dtype 是 object，resample 会报 TypeError: Only valid with DatetimeIndex,TimedeltaIndex or PeriodIndex —— 看到这个报错就是没转 datetime。

★ 前 19 行一定是 nan（窗口没满）。rolling(20).std() 默认 ddof=1（样本标准差），和 numpy 的 np.std 默认 ddof=0 不一样 —— 两套算出来的数会有细微差别。

注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。）

★ 直接 pd.concat 两个不同长度的 Series 会按索引并集对齐，缺的地方是 nan ——不会报错，但会让「月胜率」的分母变大。先 resample 到同一频率再比。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：pandas 3.0 里 'M' 已弃用，必须写 'ME'（月末）或 'MS'（月初），否则告警
- [ ] 我能不看资料写出：第一期一定是 nan（没有上一期）。填 0 会让「首月增长 0%」这个假结论进报告 —— 保留 nan，或者在报告里写「首月为基期」。
- [ ] 我能不看资料写出：dt.dayofweek 是 0=周一 … 6=周日（不是 1=周一）。dt.day 是「几号」不是「星期几」，这两个搞混会算出一堆没意义的「按星期分布」。
- [ ] 我能不看资料说出：一句话：先 to_datetime，再 set_index，最后 resample —— 顺序不能乱。
- [ ] 我能不看资料说出：注意：总收益要用累乘 (1+r).prod()-1，**不是** r.sum() ——用求和算收益率是新手最常犯的错。）
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 17 天 • apply/map/lambda + 多级索引：什么时候该用、什么时候别用

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-17-第16天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 17 天 · 11/18

applymaplambda + 多级索引：什么时候该用、什么时候别用

能用多级索引取出任意切片。

第 17 天 • 11/18 • 今日知识点回顾

apply/map/lambda + 多级索引：什么时候该用、什么时候别用

目标：判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 判断一段代码「该向量化还是该 apply」，并能用多级索引取出任意切片。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 28 分钟（边看边在本机敲一遍）

P30	Pandas2高级函数应用	约 10 分钟
P29	Pandas2数据迭代操作	约 6 分钟
P84	函数进阶之函数作为参数传递	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：多层索引的 loc / xs / IndexSlice 三种取法

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • apply 是「最后手段」，先问能不能向量化

- 一句话：能用 .str / .dt / 算术 / where 解决的，绝不用 apply。
- 代码：

```
import pandas as pd, numpy as np, pathlib, time
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
# 慢：apply + 自定义函数
t0 = time.perf_counter()
a = df['amount'].apply(lambda x: x * 0.9 if pd.notna(x) else x)
t1 = time.perf_counter()
# 快：向量化
b = df['amount'].where(df['amount'].isna(), df['amount'] * 0.9)
t2 = time.perf_counter()
print(f'apply {t1-t0:.4f}s  where {t2-t1:.4f}s  结果相同 {a.equals(b)}')
```

这一页只讲「apply 是「最后手段」，先问能不能向量化」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • apply 是「最后手段」，先问能不能向量化（续）

★ 1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。

- apply 本质是 Python 循环，向量化走的是 C。

知识点 2/7 • map：按字典做值替换，比 replace 更可控

- 一句话：s.map({'M': '男', 'F': '女'}) 找不到的变 nan —— 这一点很关键。
- 代码：

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
print(u['gender'].value_counts(dropna=False).to_dict())
m = {'M': '男', 'F': '女', '男': '男', '女': '女'}
u['g2'] = u['gender'].map(m)
print(u['g2'].value_counts(dropna=False).to_dict())
```

★ 本机实测：gender 有六种写法（男/女/M/F/未知/空），map 之后「未知」和空变成 nan。想保留成「未知」就写 m.get(x, '未知') 用 apply，或者先在字典里补 {'未知': '未知'}。map 的「找不到→nan」是隐式行为，最容易悄悄丢掉一类数据。

这一页只讲「map：按字典做值替换，比 replace 更可控」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • apply 的正确用法：一行里要看好几列

- 一句话：df.apply(func, axis=1) 把每一行当 Series 传给函数 —— 这是向量化做不到的场景。
- 代码：

知识点 3/7 • apply 的正确用法：一行里要看好几列（续）

```
def level(r):
if pd.isna(r['amount']):
return '金额缺失'
if r['amount'] >= 2000:
return '高客单'
if r['amount'] >= 500:
return '中客单'
return '低客单'
df['level'] = df.apply(level, axis=1)
print(df['level'].value_counts().to_dict())
```

★ axis=1 比 axis=0 慢得多（逐行构造 Series）。数据超过 10 万行时，改用 np.select 或 pd.cut —— 下面两条就是替代方案。

知识点 4/7 • 两个向量化替代：np.select 与 pd.cut

- 一句话：多条件分支用 np.select；等距分箱用 pd.cut；分位分箱用 pd.qcut。
- 代码：

```
cond = [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500]
df['level2'] = np.select(cond, ['金额缺失', '高客单', '中客单'], default='低客单')
print((df['level'] == df['level2']).mean())          # 1.0 完全一致
df['bin'] = pd.cut(df['amount'], bins=[0, 500, 2000, np.inf],
labels=['低', '中', '高'], right=False)
print(df['bin'].value_counts(dropna=False).to_dict())
```

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

知识点 5/7 • 多级索引：groupby 两个键之后发生了什么

- 一句话：groupby(['city','category']) 得到一个 MultiIndex，行标签是元组。
- 代码：

```
paid = df[df['status'] == 'paid']
g = paid.groupby(['city', 'category'])['amount'].sum()
print(type(g.index))                    # MultiIndex
print(g.index.names)                    # ['city', 'category']
print(g.head(3).to_string())
print(g.loc[('北京', '数码')])          # 按元组取单个值
```

★ 单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。

- 要取「北京 + 数码」必须写成元组 ('北京', '数码')。

这一页只讲「多级索引：groupby 两个键之后发生了什么」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • xs 与 IndexSlice：取某一层的切片

- 一句话：g.xs('数码', level='category') 取所有城市在数码上的值。
- 代码：

```
print(g.xs('数码', level='category').sort_values(ascending=False).to_string())
print(g.loc[(slice(None), '数码'), ].sort_values(ascending=False).head(3).to_string())
idx = pd.IndexSlice
print(g.loc[idx[:, '数码']].head(3).to_string())
```

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

知识点 7/7 • reset_index / unstack / stack：三层索引的三个出口

- 一句话：reset_index 变平表、unstack 变宽表、stack 变长表。
- 代码：

```
print(g.reset_index().head(3).to_string(index=False))   # 变回普通列
print(g.unstack(fill_value=0).shape)                    # (9, 4) 宽表
print(g.unstack().stack().shape)                        # 又变回来（nan 会被丢）
```

- （stack() 默认 dropna=True，会**丢掉** nan 的组合 ——unstack().stack() 不一定还原成原样。要严格还原写 stack(dropna=False)。）

代码速查：今天这几段，要在本机跑通（1/7）

写 apply_practice.py，同时验证「两种写法结果一致」：

```
import pandas as pd, numpy as np, pathlib, time
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) apply vs 向量化 计时

t0 = time.perf_counter()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/7）

```
a = df['amount'].apply(lambda x: round(x * 0.9, 2) if pd.notna(x) else x)
```

t1 = time.perf_counter()

b = (df['amount'] * 0.9).round(2)

t2 = time.perf_counter()

```
print(f'apply {t1-t0:.4f}s  vector {t2-t1:.4f}s  equal={a.equals(b)}')
```

```
# 2) 分档：apply / np.select / pd.cut 三种写法对齐
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/7）

```
def level(r):
```

```
    if pd.isna(r['amount']): return '缺失'
    if r['amount'] >= 2000: return '高'
    if r['amount'] >= 500: return '中'
    return '低'
```

```
df['lv_apply'] = df.apply(level, axis=1)
```

```
df['lv_select'] = np.select(
```

代码速查：今天这几段，要在本机跑通（4/7）

```
    [df['amount'].isna(), df['amount'] >= 2000, df['amount'] >= 500],
```

['缺失', '高', '中'], default='低')

```
print('apply==select: ', (df['lv_apply'] == df['lv_select']).mean())
```

```
print(df['lv_select'].value_counts().to_dict())
```

3) map 归并 gender

```
u = pd.read_csv(DATA / 'users_dirty.csv', encoding='utf-8-sig')
```

代码速查：今天这几段，要在本机跑通（5/7）

```
m = {'M': '男', 'F': '女', '男': '男', '女': '女', '未知': '未知'}
```

u['g2'] = u['gender'].map(m)

```
print('map 后分布: ', u['g2'].value_counts(dropna=False).to_dict())
```

```
print('map 丢掉的行: ', u['g2'].isna().sum())
```

4) 多级索引三种取法

paid = df[df['status'] == 'paid']

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/7）

```
g = paid.groupby(['city', 'category'])['amount'].sum()
```

```
print(g.xs('数码', level='category').sort_values(ascending=False).head(3).to_string())
```

```
print(g.reset_index().shape, g.unstack(fill_value=0).shape)
```

验收（做到才算过关）：

① 第 1 题 equal=True，并报出两种写法的耗时比；

② 第 2 题 apply 与 np.select 结果 100% 一致；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/7）

```
③ 第 3 题能报出 map 之后有多少行变 nan（就是「未知」+ 空的那些）。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ np.select 的条件是「从上往下第一个为 True 的生效」，顺序错了结果就错 ——和 if/elif 一样的规则。pd.cut 的 right=False 表示左闭右开 [0,500)，默认 right=True 是 (0,500]，边界值会掉进不同的桶。

★ 三种写法等价，但 xs 最短、最不容易写错。IndexSlice 适合「同时切两层」的复杂场景（比如 loc[idx['北京':'上海', '数码':'家居']]）。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：1163 行时 apply 可能只慢 3–5 倍，看着无所谓；到 100 万行时是 50 倍以上。
- [ ] 我能不看资料写出：单个键用 g.loc['北京'] 也能取，但会返回一个 Series（4 个品类）。
- [ ] 我能不看资料说出：要取「北京 + 数码」必须写成元组 ('北京', '数码')。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 18 天 • 可视化：matplotlib 中文设置 + 三张有业务含义的图

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-18-第17天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

分析期 · 第 18 天 · 11/19

可视化：matplotlib 中文设置 + 三张有业务含义的图

能配一句话结论，中文字体不乱码。

第 18 天 • 11/19 • 今日知识点回顾

可视化：matplotlib 中文设置 + 三张有业务含义的图

目标：产出 3 张图（趋势 / 结构 / 对比），每张图能配一句话结论，中文字体不乱码。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 产出 3 张图（趋势 / 结构 / 对比），每张图能配一句话结论，中文字体不乱码。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 42 分钟（边看边在本机敲一遍）

P3	matplotlib折线图基本使用	约 16 分钟
P9	matplotlib常用统计图	约 12 分钟
P11	matplotlib直方图	约 14 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst`

第二章末尾：Series.plot / DataFrame.plot 的常用参数

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 中文字体：一行设置，否则全是方框

- 一句话：plt.rcParams['font.sans-serif'] = ['Microsoft YaHei'] + axes.unicode_minus=False。
- 代码：

```
import matplotlib
matplotlib.use('Agg')          # 不弹窗，直接存文件
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 120
```

★ 不设 font.sans-serif 时中文全是方框（MissingGlyphWarning）；

- 不设 axes.unicode_minus=False 时负号也是方框 —— 画带负值的折线必踩。

这一页只讲「中文字体：一行设置，否则全是方框」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注

- 一句话：折线图回答「在变好还是变坏」，一定要标出最高点和最低点。
- 代码：

这一页只讲「图 1 趋势图：月度实付折线 + 标注」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注（续）

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
paid = df[df['status'] == 'paid'].set_index('create_time')
m = paid['amount'].resample('ME').sum()
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(m.index.strftime('%Y-%m'), m.values, marker='o', color='#0F2E35')
ax.set_title('月度实付金额（paid 订单）')
ax.set_ylabel('元')
ax.grid(alpha=.3)
top = m.idxmax()
ax.annotate(f'最高 {m.max():,.0f}', xy=(m.index.get_loc(top), m.max()),
xytext=(0, 12), textcoords='offset points', ha='center', color='#C0392B')
fig.tight_layout(); fig.savefig(DATA / 'fig1_trend.png'); plt.close(fig)
```

这一页只讲「图 1 趋势图：月度实付折线 + 标注」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • 图 1 趋势图：月度实付折线 + 标注（续）

> ★ 本机数据只有 3 个月（2026-08 / 09 / 10），折线只有 3 个点 ——这种时候**不要**画折线，画柱状图更诚实。图表选型跟着数据量走。

知识点 3/7 • 图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大

- 一句话：堆叠条形回答「总量由哪几块组成」，横向条形比竖向更好读中文标签。
- 代码：

这一页只讲「图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大」。讲完就翻页，下一页是另一个知识点。

知识点 3/7 • 图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大（续）

```
pv = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc='sum', fill_value=0)
pv = pv.loc[pv.sum(axis=1).sort_values().index]      # 按总量排序
fig, ax = plt.subplots(figsize=(8, 5))
pv.plot(kind='barh', stacked=True, ax=ax, width=.72)
ax.set_title('各城市实付构成（按品类）')
ax.set_xlabel('元'); ax.legend(title='品类', fontsize=9)
fig.tight_layout(); fig.savefig(DATA / 'fig2_structure.png'); plt.close(fig)
```

★ 横向条形（barh）时 y 轴是从下往上排，所以要先 sort_values() 升序，画出来才是「最大的在最上面」。忘了排序 = 图看着乱但没错，最难发现。

这一页只讲「图 2 结构图：城市 × 品类堆叠条形，看谁贡献最大」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 图 3 对比图：漏斗或分布，二选一

- 一句话：对比图回答「差多少」，用条形；看分布形状，用直方图。
- 代码：

知识点 4/7 • 图 3 对比图：漏斗或分布，二选一（续）

```
e = pd.read_csv(DATA / 'events_dirty.csv', encoding='utf-8-sig')
step = ['view', 'cart', 'order', 'pay']
cnt = e.groupby('event')['user_id'].nunique().reindex(step)
fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(step, cnt.values, color=['#0F2E35', '#2E7D8A', '#5BA3AE', '#9CC9D0'])
for i, v in enumerate(cnt.values):
ax.text(i, v, f'{v:,}\n{v/cnt.iloc[0]*100:.1f}%', ha='center', va='bottom', fontsize=9)
ax.set_title('用户转化漏斗（去重用户数）')
ax.set_ylim(0, cnt.max() * 1.25)
fig.tight_layout(); fig.savefig(DATA / 'fig3_funnel.png'); plt.close(fig)
```

★ 本机实测：view 500 / cart 150 / order 74 / pay 28，漏斗率 100% / 30.0% / 14.8% / 5.6% —— 这四个数要能和 events 表对得上。

这一页只讲「图 3 对比图：漏斗或分布，二选一」。讲完就翻页，下一页是另一个知识点。

知识点 5/7 • figsize 与 dpi：进简历的图要多大

- 一句话：figsize=(8,4) + dpi=150 → 1200×600 px，贴进 Word 不糊。
- 代码：
- fig, ax = plt.subplots(figsize=(8, 4), dpi=150)
- fig.savefig(DATA / 'x.png', bbox_inches='tight') # 裁掉多余白边

★ 不写 bbox_inches='tight'，图四周会留一大圈白 —— 贴进文档后看着很小。

- 这是「图看着糊/小」最常见的原因，不是 dpi 不够。

知识点 6/7 • 三张图的结论怎么写

- 一句话：每张图配一句话，格式是「谁 / 多少 / 所以呢」。
- 代码：

```
CONCL = {
'fig1_trend.png': '8–10 月实付逐月变化，最高月是 10 月，说明活动集中在季末。',
'fig2_structure.png': '南京实付 60,532 排第一，其中数码占大头，家居偏弱。',
'fig3_funnel.png': '浏览到加购转化 30%，加购到下单只有 14.8%，卡点在加购后。',
```

- }

- （结论里必须出现数字。「增长明显」「表现较好」这类词在面试里等于没说话。）

这一页只讲「三张图的结论怎么写」。讲完就翻页，下一页是另一个知识点。

知识点 7/7 • 图别只存 PNG，也存一份数据

- 一句话：图会过时，数据不会 —— 每张图旁边存一份它用的 CSV。
- 代码：
- pv.round(2).to_csv(DATA / 'fig2_data.csv', encoding='utf-8-sig')
- cnt.rename('users').to_csv(DATA / 'fig3_data.csv', encoding='utf-8-sig')

★ 面试官问「这张图的数据哪来的」，能当场打开 CSV 比能打开 PPT 有说服力。

代码速查：今天这几段，要在本机跑通（1/9）

```
写 viz_three.py，产出 3 张 PNG + 3 份数据 CSV + 一段结论：
```

```
import pathlib
```

```
import matplotlib
```

matplotlib.use('Agg')

```
import matplotlib.pyplot as plt
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/9）

```
import pandas as pd
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
```

```
plt.rcParams['axes.unicode_minus'] = False
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
OUT = DATA / 'figs'; OUT.mkdir(exist_ok=True)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/9）

图1 月度实付（3 个月 → 用柱状，不用折线）

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig', parse_dates=['create_time'])
```

```
paid = df[df['status'] == 'paid'].set_index('create_time')
```

m = paid['amount'].resample('ME').sum()

```
fig, ax = plt.subplots(figsize=(7, 4), dpi=150)
```

```
ax.bar(m.index.strftime('%Y-%m'), m.values, color='#0F2E35', width=.5)
```

代码速查：今天这几段，要在本机跑通（4/9）

```
for i, v in enumerate(m.values):
```

```
    ax.text(i, v, f'{v:,.0f}', ha='center', va='bottom', fontsize=9)
```

```
ax.set_title('月度实付金额（paid）'); ax.set_ylabel('元')
```

ax.set_ylim(0, m.max() * 1.15)

```
fig.tight_layout(); fig.savefig(OUT / 'fig1_month.png', bbox_inches='tight'); plt.close(fig)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/9）

```
m.round(2).to_csv(OUT / 'fig1_data.csv', encoding='utf-8-sig')
```

图2 城市 × 品类 堆叠（横向）

```
pv = pd.pivot_table(paid, index='city', columns='category',
```

```
                    values='amount', aggfunc='sum', fill_value=0)
```

```
pv = pv.loc[pv.sum(axis=1).sort_values().index]
```

代码速查：今天这几段，要在本机跑通（6/9）

```
fig, ax = plt.subplots(figsize=(8, 5), dpi=150)
```

```
pv.plot(kind='barh', stacked=True, ax=ax, width=.72)
```

```
ax.set_title('各城市实付构成'); ax.set_xlabel('元')
```

```
fig.tight_layout(); fig.savefig(OUT / 'fig2_city.png', bbox_inches='tight'); plt.close(fig)
```

```
pv.round(2).to_csv(OUT / 'fig2_data.csv', encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/9）

图3 漏斗

```
e = pd.read_csv(DATA / 'events_dirty.csv', encoding='utf-8-sig')
```

step = ['view', 'cart', 'order', 'pay']

```
cnt = e.groupby('event')['user_id'].nunique().reindex(step)
```

```
fig, ax = plt.subplots(figsize=(7, 4), dpi=150)
```

```
ax.bar(step, cnt.values, color=['#0F2E35', '#2E7D8A', '#5BA3AE', '#9CC9D0'])
```

代码速查：今天这几段，要在本机跑通（8/9）

```
for i, v in enumerate(cnt.values):
```

```
    ax.text(i, v, f'{v:,}\n{v/cnt.iloc[0]*100:.1f}%', ha='center', va='bottom', fontsize=9)
```

```
ax.set_title('用户转化漏斗'); ax.set_ylim(0, cnt.max() * 1.3)
```

```
fig.tight_layout(); fig.savefig(OUT / 'fig3_funnel.png', bbox_inches='tight'); plt.close(fig)
```

```
cnt.to_csv(OUT / 'fig3_data.csv', encoding='utf-8-sig')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（9/9）

验收（做到才算过关）：

① 三张 PNG 都能打开、中文不乱码、负号（如有）不是方框；

```
② fig3 的四个数 = 500 / 150 / 74 / 28，漏斗率 100% / 30.0% / 14.8% / 5.6%；
```

③ 每张图能配一句带数字的结论，念出来不超过 25 个字。

最容易错的地方

★ 横向条形（barh）时 y 轴是从下往上排，所以要先 sort_values() 升序，画出来才是「最大的在最上面」。忘了排序 = 图看着乱但没错，最难发现。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：本机数据只有 3 个月（2026-08 / 09 / 10），折线只有 3 个点 ——这种时候**不要**画折线，画柱状图更诚实。图表选型跟着数据量走。
- [ ] 我能不看资料写出：不写 bbox_inches='tight'，图四周会留一大圈白 —— 贴进文档后看着很小。
- [ ] 我能不看资料写出：面试官问「这张图的数据哪来的」，能当场打开 CSV 比能打开 PPT 有说服力。
- [ ] 我能不看资料说出：一句话：折线图回答「在变好还是变坏」，一定要标出最高点和最低点。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 19 天 • ★Excel 进阶：Power Query 清洗 + 条件格式 + 图表；下午开投

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-19-第18天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data